

Programmation Orientée Objet (POO)

🔧 Le Constructeur et l'Instance

La **classe** est le plan de construction. L'**instance** est l'objet réel créé à partir de ce plan.

Le **constructeur** (méthode `__init__`) est appelé automatiquement lors de la **création d'une instance** pour préparer l'objet.

```
root@python:~$
```

```
class Robot:
    def __init__(self, nom, version=1.0): # Constructeur
        self.nom = nom                  # Attribut passé en par
        self.version = version          # Attribut avec valeur
        self.batterie = 100             # Attribut par défaut (
```

Création d'une instance (instanciation)

```
mon_robot = Robot("R2D2")
```

📊 Les Attributs : Accès et Modification

Un **attribut** est une variable interne à l'objet qui définit son état. On utilise la notation pointée `.` pour agir dessus.

```
root@python:~$
```

```
# Accès à un attribut
print(mon_robot.nom) # Affiche "R2D2"
```

Modification d'un attribut

```
mon_robot.version = 2.0
print(mon_robot.version) # Affiche 2.0
```

⚙️ Les Méthodes et leur Appel

Une **méthode** est une fonction définie à l'intérieur d'une classe. Elle représente un comportement de l'objet. Le paramètre `self` permet à la méthode d'accéder aux attributs de l'objet qui l'appelle.

```
root@python:~$
```

```
class Robot:
    # ... (constructeur précédent) ...

    def saluer(self): # Définition d'une méthode
        print(f"Bonjour, je suis {self.nom}")

    def charger(self, energie):
        self.batterie += energie
```

Appel de méthode

```
mon_robot.saluer()      # Affiche "Bonjour, je suis R2D2"
mon_robot.charger(20)   # Modifie l'état interne (batterie)
```

🧠 À retenir

- **self** : représente l'instance sur laquelle on travaille. Il doit être le 1er paramètre de chaque méthode.
- **Attribut par défaut** : peut être défini dans les arguments de `__init__` (ex: `version=1.0`) ou directement dans le corps du constructeur (ex: `self.batterie = 100`).
- **Encapsulation** : regrouper les données (attributs) et les fonctions (méthodes) au sein d'une même entité.

[Image showing the relationship between a class and an instance in object oriented programming]

FOLLOW
THE
PYTHONIC
WAY

HACK
AGAINST
MACHISM

